



UNIVERSITAT
ROVIRA I VIRGILI
Fundació URV



Manifesting the Elasticity of Serverless Data Pipelines: a Metabolomics Use Case

A Serverless Implementation of a Production-Grade Metabolite Annotation Pipeline

Elena Díez-Cuadrado, URV

Germán T. Eizaguirre, URV

Enrique Molina-Giménez, URV

WoSC11 – 11th International Workshop on Serverless Computing



Elena Díez-Cuadrado

- **Undergraduate student** at Universitat Rovira i Virgili (URV)
- Currently pursuing a Double Bachelor's Degree in **Biotechnology & Computer Engineering**
- **Research intern** at Cloud & Distributed Systems Lab, URV

Co-authors

- **Germán T. Eizaguirre, CloudLab, URV**
- **Enrique Molina-Giménez, CloudLab, URV**

Research group - CloudLab

- **Cloud & Distributed Systems Lab (CloudLab), URV**
- Led by **Prof. Pedro García-López**
- Serverless platforms, cloud and edge computing, data-intensive systems

Table of contents

01

METASPACE
platform

02

METASPACE
pipeline

03

Lithops
Implementation

04

Scaling with
database size

05

Scaling with
dataset size

06

Ingestion
optimization

07

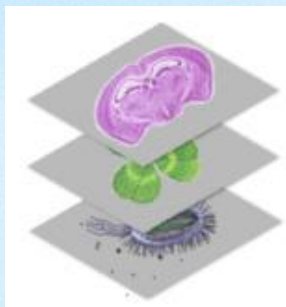
Takaways and
future work

01

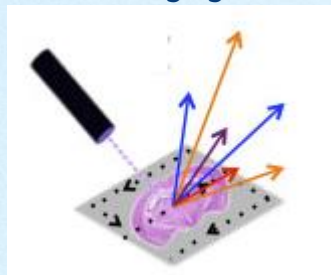
The METASPACE Platform

Spatial metabolomics in practice: METASPACE

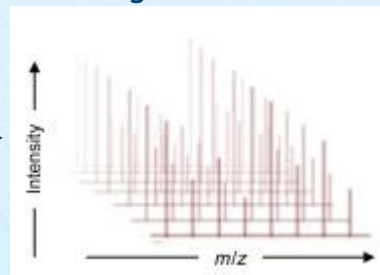
Sample preparation



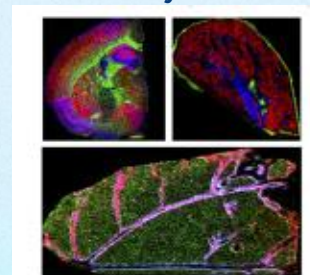
Mass spectrometry imaging



Molecular image generation



Data processing and analysis



└─ imzML files ─┐

Data processing: Metabolite annotation pipeline



METASPACE

The platform for metabolite annotation of imaging mass spectrometry data



Metabolite annotation

[Submit](#) your high-resolution imaging mass spectrometry data to our high-throughput metabolite annotation engine.



Explore the knowledgebase

[Browse](#) annotations from all datasets using our interactive interface. Search and compare your annotations with those from the community.

- Spatial metabolomics = **where** metabolites are
- Imaging Mass Spectrometry → A 2D map of molecules in tissue
- **METASPACE**: open platform for metabolite annotation of imaging MS data
- **Hundreds of terabytes** of public data, growing every day
- Each dataset: raw size in GB, but **internally expanded from 10x to 100x** during analysis

02

Metabolite Annotation Pipeline

The METASPACE metabolomics pipeline

Raw imzML + ibd files



Annotated ion images

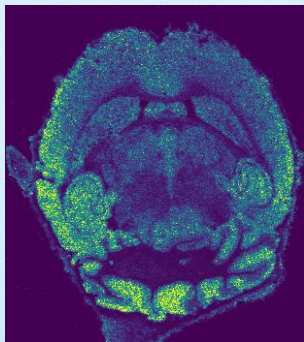


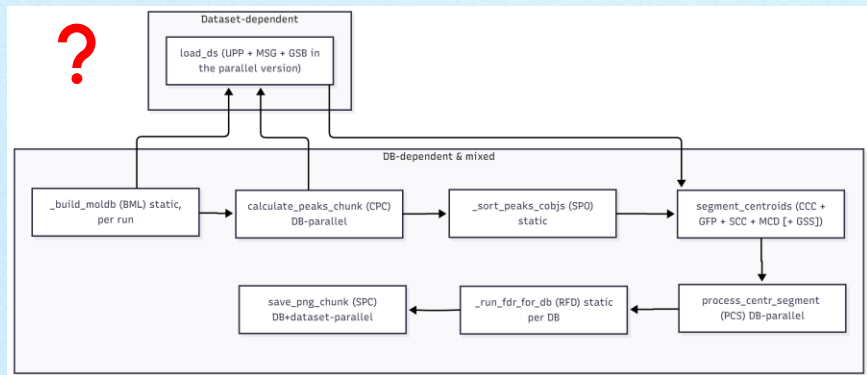
Table 1. Classification of pipeline stages

Category	Stage	Acronym
Database-dependent	calculate_peaks_chunk	CPC
	process_centri_segment	PCS
Dataset-dependent	upload_partitions	UPP
	merge_segments	MSG
Dependent on both	merge_centri_df_segments	MCD
	save_png_chunk	SPC
	get_segmi_stats	GSS
Static	_build_molddb	BML
	_run_fdr_for_db	RFD
	_sort_peaks_cobjs	SPO
	batch_copy	BCP
	clip_centri_df_chunk	CCC
	get_first_peak_mz	GFP
	get_segmi_bounds	GSB
	segment_centri_chunk	SCC

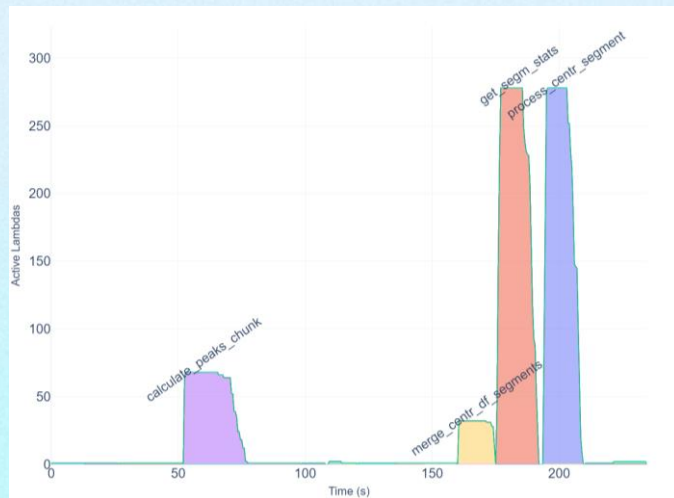
Motivation: why serverless here?

Our pipeline: multi-stage DAG

- Each stage has different **parallelism and resource needs**
- **Hard to pre-size clusters** for every dataset
- Existing serverless work: microbenchmarks, few complex real pipelines



How big will each stage be?



Concurrency of active lambdas by stage for a single 50 MB dataset run. Parallelism changes from tens of functions to almost 300.

03

Our Implementation

Serverless Implementation: Lithops + PyRun

- Framework: Lithops – multi-cloud serverless analytics in Python
- Backend: AWS Lambda + AWS S3 for storing input and intermediate data
- Orchestration: PyRun – managed serverless Python environment



PyRun Effortless Cloud Computing for Python

Run scalable Python workloads for data processing, AI, and distributed computing without managing complex cloud infrastructure. Focus on your code, not the setup. Currently supporting AWS, with GCP and Azure planned.



AWS Lambda



Amazon S3
Simple Storage Service

PyRun.cloud

PyRun Platform

Frameworks

Lithops (FaaS)
Dask (Clusters)
Ray
Cubed

Multi-Cloud Support

aws
IBM Cloud
Open Nebula
On Premise Clusters
Google Cloud

Multi-Backend Support

Managed
Kubernetes
Lambda
VMs
GPU Clusters
S3
OpenWhisk

Automated Runtime
Management
Dataplug (On-the-fly
Partitioning)
Ready to Run Pipelines

VSCoDe Interface

PyRun Studio

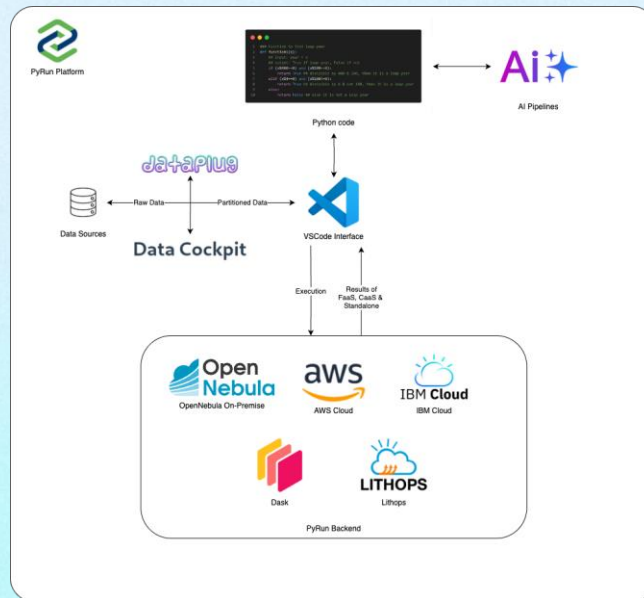
Development & Workflow

Workspaces
Data Cockpit
Real-Time Monitoring
Colaborative Features

AI & ML Tools

AI Code Assistant
HuggingFace Model
Hub
RAG (Retrieval-
Augmented Generation)
MCP (Model Context
Protocol)

Integrated & Automated



Experimental setup

Two main experiments: varying database size (fixed dataset) & varying dataset size (fixed database)

Dataset	Size	Reference DB	# Chunks	# Segments
CT26_xenograft	1.8 GB	500 formulas	15	8
CT26_xenograft	1.8 GB	12,000 formulas	15	8

Table 2. Setup of the experiments with **varying database size**.

Dataset	Size	Reference DB	# Chunks	# Segments
CT26_xenograft	1.8 GB	12,000 formulas	15	8
Mousebrain test	3,9 GB	12,000 formulas	56	18
X089 Mousebrain	7 GB	12,000 formulas	100	50

Table 3. Setup of the experiments with **varying dataset size**.

04

Experiment 1: Scaling with Database Size

Elasticity: scaling with database size

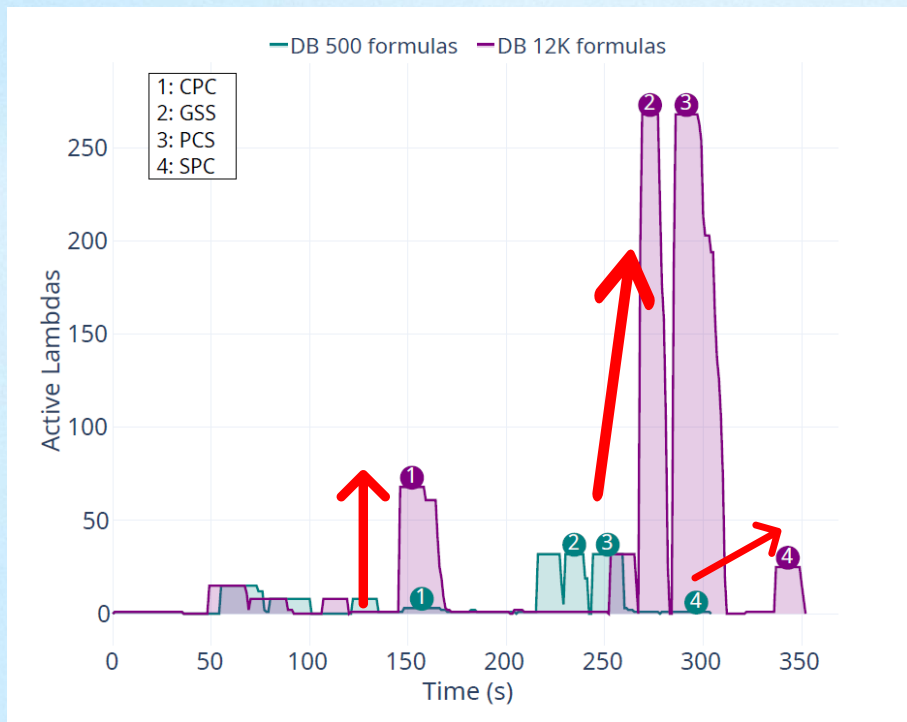


Figure 1. Concurrency of **active AWS Lambda** functions over time, highlighting the stages most influenced by database size.

- Fixed dataset: **1,8 GB**
- Isn't that **too small?**: in **PCS**, the **aggregate volume of processed data expands to 100GB!**
- DB size: 500 → 12,000 formulas
- **Elasticity** (number of active lambdas per stage):
CPC: from **~10 to ~75**
GSS & PCS: from **~35 to ~270**
SPC: from **~5 to ~30**

Less than 50 secs between both executions ends.

Elasticity: scaling with database size

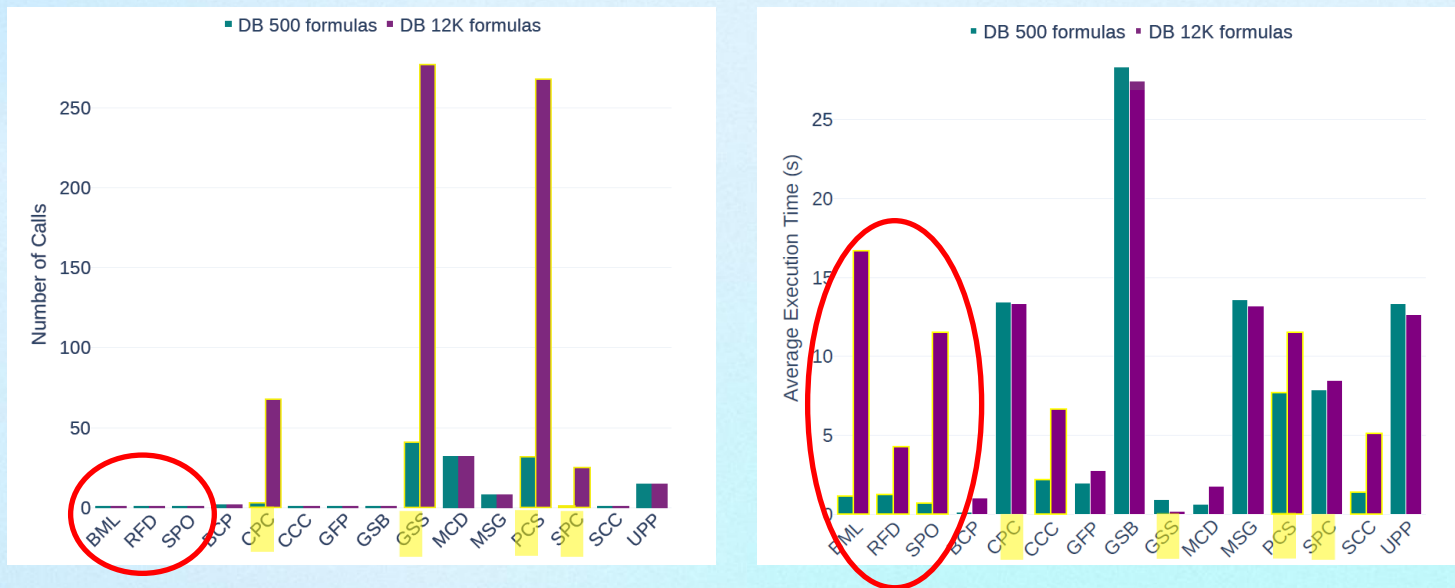


Figure 2. Comparison of (left) **function call numbers** and (right) **stage execution times** for two increasing database sizes.

For the database-influenced stages (indicated in yellow, PC, GSS, PCS and SPC), we observe that although the parallelism increases, **average execution times remain stable**.

CPU and memory under database growth

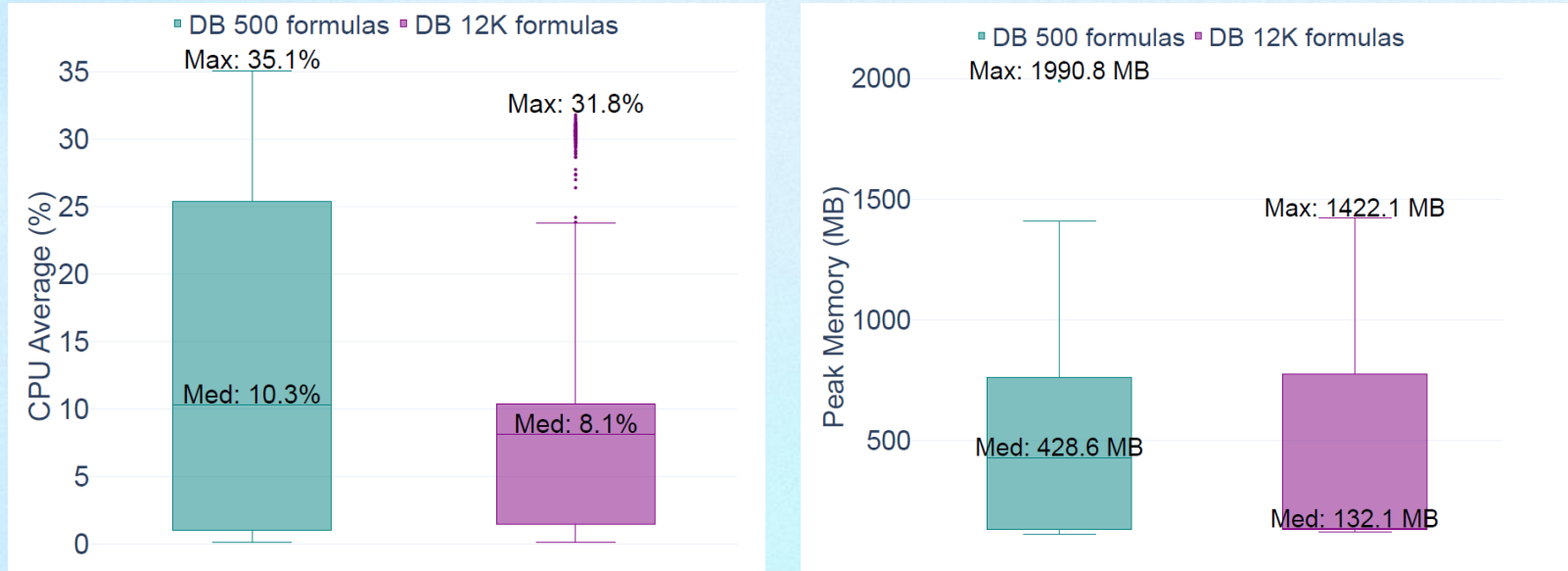


Figure 3. Comparison of (left) **CPU** and (right) **memory usage distributions** across different sizes in database-influenced stages (CPC, GSS, PCS, and SPC).

Database-influenced stages keep **low CPU (~8-10%)** and **reduced median memory** as the DB grows, indicating that **elasticity** is achieved **without saturating resources**.

05

Experiment 2: Scaling with Dataset Size

Elasticity: scaling with dataset size



Figure 4. Concurrency of **active AWS Lambda** functions over time for three dataset sizes, highlighting the stages influenced the most by dataset size.

- Fixed database: **12k formulas**
 - DS size: 1,8 GB, 3,9 GB, 7 GB
 - Number of active lambdas per stage:
MSG: from **~20** to **~55**
UPP: from **~30** to **~100**
- End-to-end time is way different in the three executions.**

Elasticity: scaling with dataset size

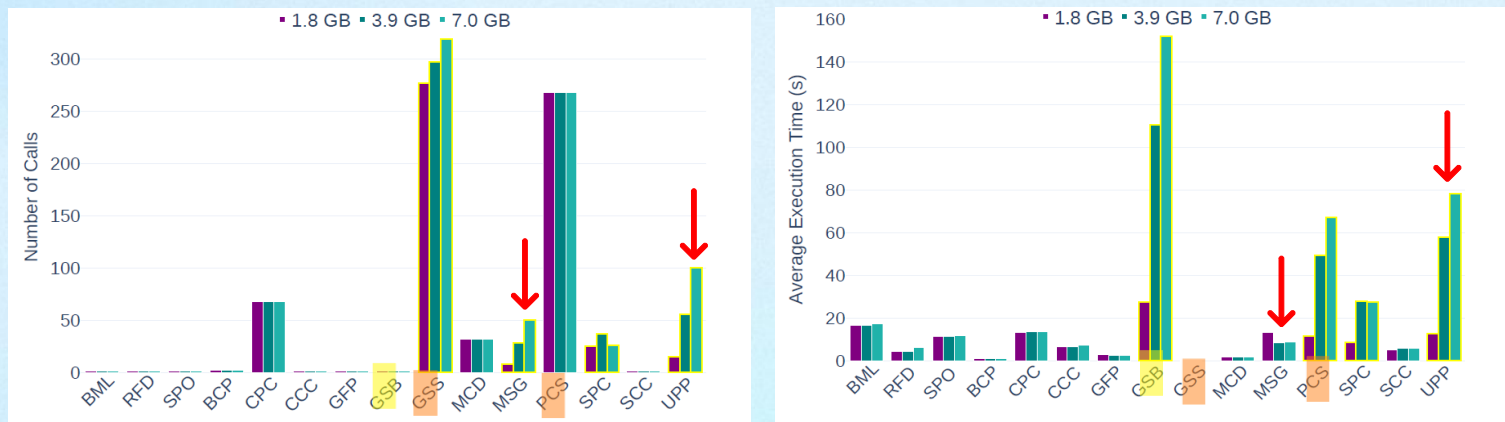


Figure 5. Comparison of (left) **function call numbers** and (right) **stage execution times** as dataset sizes increases

UPP & MSG (ingestion): both **function calls AND stage time grow** with dataset size -> **limited elasticity**
GSS vs PCS: GSS: many more **lightweight** calls; PCS: noticeable **increase** in average time (**I/O-bound**)
GSB (static, sequential): becomes a **major bottleneck**, largest increase in average execution time

CPU and memory under dataset growth

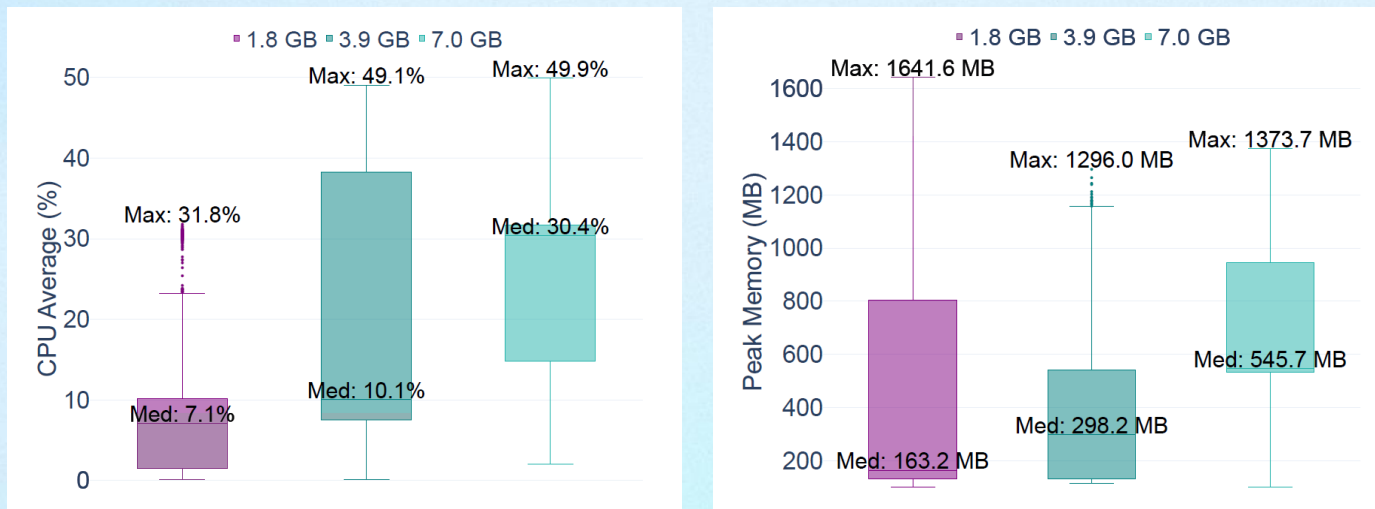


Figure 6. Comparison of (left) **CPU** and (right) **memory usage distributions** across different dataset sizes in dataset-stages (UPP, MSG).

CPU per Lambda: median rises from ~7% (1.8 GB) to ~30% (7 GB)

Memory per Lambda: median grows from ~160 MB to ~550 MB for the largest dataset
Dataset-driven ingestion stages **do not keep resource usage stable** → room for further scaling optimizations

06

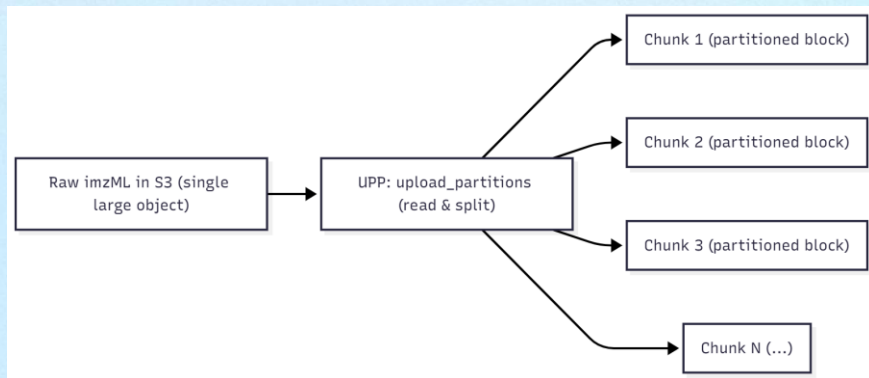
The Virtue of a Cloud- Optimized Data Ingestion

Data Ingestion: Optimization Target

Ingestion = distributed sort: upload_partitions (UPP) + merge_segments (MSG)
Choosing the best number of chunks (=number of Lambda functions) is non-trivial
Can we tune this decision heuristically?

- **DataPlug**: cloud-aware, format-aware partitioning of scientific data from object storage
- **DataCockpit**: interactive UI for DataPlug to run benchmarks, test partition strategies and explore datasets

Idea: benchmark UPP in isolation, then transfer optimal settings back to full pipeline



Data Cockpit
Efficiently manage and explore your data

Benchmarking: Enabled
Disabled
Enabled

Minimum Batch Size: 1
Maximum Batch Size: 100
Step Size: 10

Run Benchmarking

Methodology

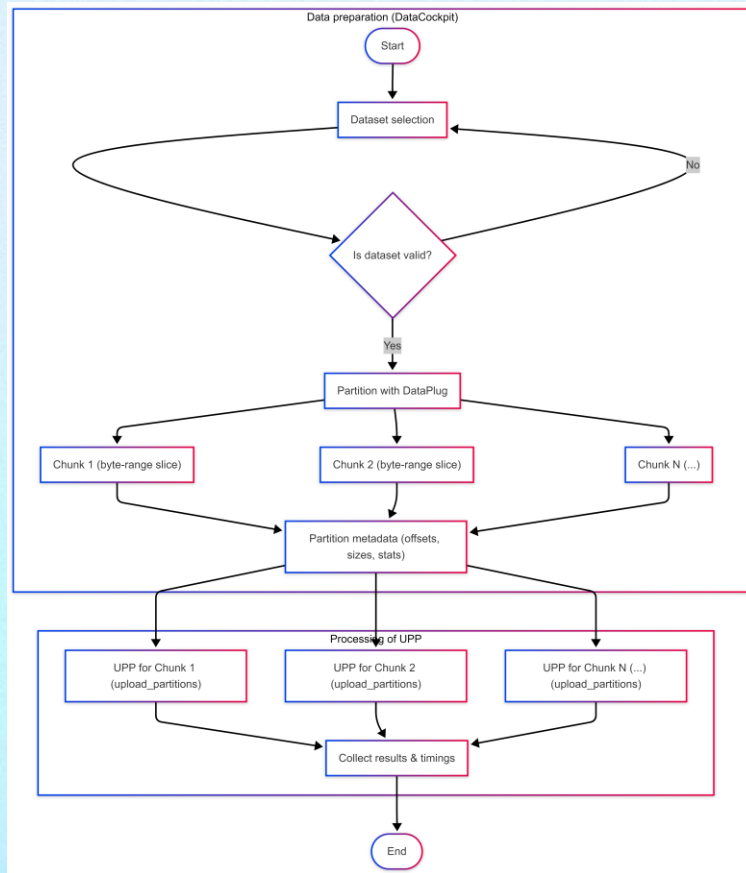
Phase 1: isolate UPP as a benchmarkable function



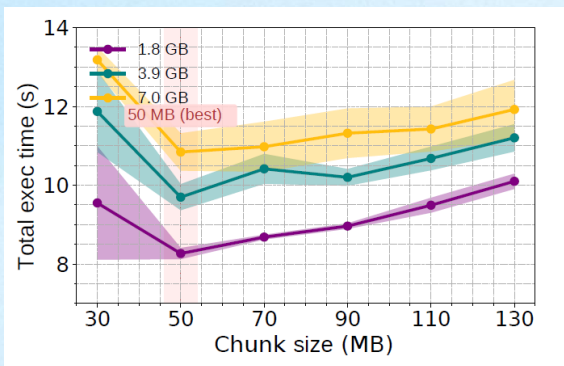
Phase 2: DataCockpit benchmarks UPP with chunk sizes 30-130 MB on 3 dataset sizes



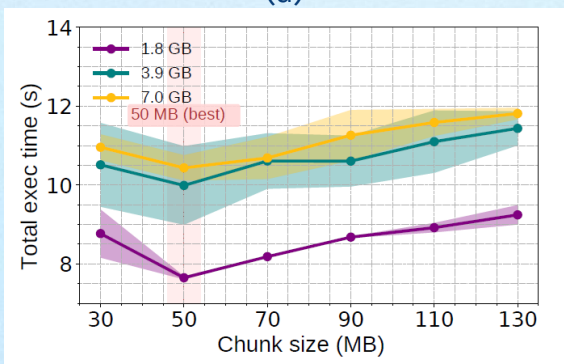
Phase 3: validate by running the original UPP and comparing execution times



Optimized Ingestion with a 50 MB Chunk Size



(a)



(b)

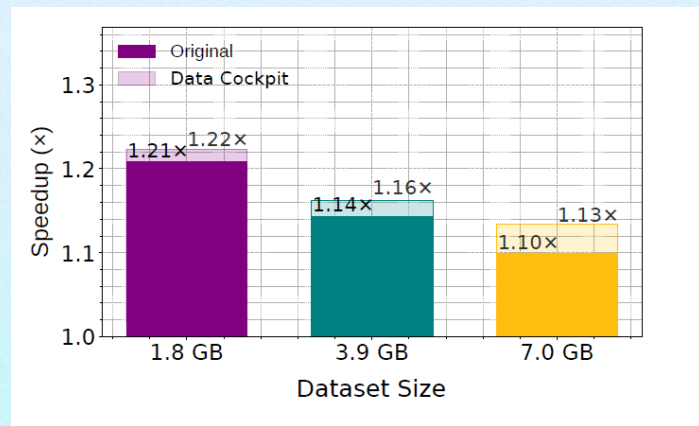


Figure 8. Speedup of UPP with an optimal chunk size of 50 MB, inferred with Data Cockpit, against a 128 MB chunk size

Figure 7. Ingestion benchmarking of UPP with (a) Data Cockpit and (b) pipeline validation

07

Conclusions

Takeaways and Future Work

Motivation

Spatial metabolomics: extreme data, non-expert users

Solution and findings

- Lithops + PyRun: fully serverless METASPACE, hundreds of parallel functions
- Elasticity: scales for DB growth; limited by DS size and sequential I/O stages
- DataCockpit + DataPlug: tune ingestion and pick best chunk size

Problem

- Complex multi-stage DAGs with highly variable parallelism
- Hard to provision and tune resources manually

Future work

- Reduce sequential bottlenecks
- Improve memory efficiency at larger scales
- Explore parallelism optimizations for other pipeline stages

Thank you!

Do you have any questions?

CONTACT:

elena.diez@urv.cat

www.linkedin.com/in/elena-diezz